3. Introduction to mCRL2

José Proença

System Verification (CC4084) 2026/2027

CISTER – U.Porto, Porto, Portugal

https://fm-dcc.github.io/sv2627





mCRL2

`http://mcrl2.org`

- Formal specification language with an associated toolset
- Used for modelling, validating and verifying concurrent systems and protocols
- Tool suggestion: use `mcrl2ide` (not `mcrl2-gui`)

Recall CCS semantics

$$\frac{}{\alpha.P \xrightarrow{\alpha} P}\ (\text{act})$$

$$\frac{P_1 \xrightarrow{\alpha} P_1'}{P_1 + P_2 \xrightarrow{\alpha} P_1'}\ (\text{sum-1})$$

$$\frac{P_2 \xrightarrow{\alpha} P_2'}{P_1 + P_2 \xrightarrow{\alpha} P_2'}\ (\text{sum-2})$$

$$\frac{P \xrightarrow{\alpha} P'}{P\backslash L \xrightarrow{\alpha} P'\backslash L}\ \alpha \notin L\ (\text{res})$$

$$\frac{P \xrightarrow{\alpha} P'}{P[f] \xrightarrow{f(\alpha)} P'[f]}\ (\text{rel})$$

$$\frac{K = P \quad P \xrightarrow{\alpha} P'}{K \xrightarrow{\alpha} P'}\ (\text{rec})$$

$$\frac{P \xrightarrow{\alpha} P'}{P|Q \xrightarrow{\alpha} P'|Q}\ (\text{com1})$$

$$\frac{Q \xrightarrow{\alpha} Q'}{P|Q \xrightarrow{\alpha} P|Q'}\ (\text{com2})$$

$$\frac{P \xrightarrow{a} P' \quad Q \xrightarrow{\bar{a}} Q'}{P|Q \xrightarrow{\tau_a} P'|Q'}\ (\text{com3})$$

Processes in mCRL2

Syntax (by example)

$$
\begin{aligned}
a.\mathbf{0} &\longrightarrow \texttt{a} \\
a.P &\longrightarrow \texttt{a.P} \\
P_1 + P_2 &\longrightarrow \texttt{P1 + P2} \\
P\backslash L &\longrightarrow \texttt{block(L,P)} \\
P[f] &\longrightarrow \texttt{rename(f,P)} \\
a.P|\overline{a}.Q &\longrightarrow \texttt{comm(\{a1|a2->a\},a1.P||a2.Q)} \\
a.P|\overline{a}.Q\backslash\{a\} &\longrightarrow \texttt{block(\{a1,a2\},comm(\{a1|a2->a\},} \\
&\qquad \texttt{a1.P||a2.Q))}
\end{aligned}
$$

CCS in mCRL2 hiding communication

Syntax (by example)

$$
\begin{aligned}
a.\mathbf{0} &\rightarrow \texttt{a} \\
a.P &\rightarrow \texttt{a.P} \\
P_1 + P_2 &\rightarrow \texttt{P1 + P2} \\
P\backslash L &\rightarrow \texttt{block(L,P)} \\
P[f] &\rightarrow \texttt{rename(f,P)} \\
a.P|\overline{a}.Q &\rightarrow \texttt{hide(\{a\},comm(\{a1|a2->a\},a1.P||a2.Q))} \\
a.P|\overline{a}.Q\backslash\{a\} &\rightarrow \texttt{hide(\{a\},block(\{a1,a2\},comm(\{a1|a2->a\},} \\
&\qquad \texttt{a1.P||a2.Q)))}
\end{aligned}
$$

mCRL2

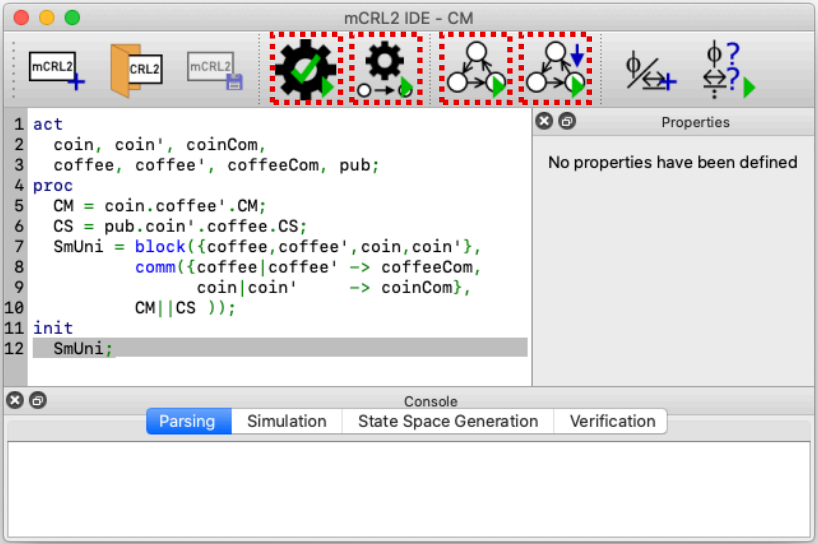
$$CM = \text{coin}.\overline{\text{coffee}}.CM$$

$$CS = \text{pub}.\overline{\text{coin}}.\text{coffee}.CS$$

$$SmUni = (CM|CS)\backslash\{\text{coin}, \text{coffee}\}$$

```
act
  coin, coin', coinCom,
  coffee, coffee', coffeeCom, pub;
proc
  CM = coin.coffee'.CM;
  CS = pub.coin'.coffee.CS;
  SmUni = block({coffee,coffee',coin,coin'},
          comm({coffee|coffee' -> coffeeCom,
                coin|coin'     -> coinCom},
          CM||CS ));
init
  SmUni;
```

mCRL2 IDE



Parse

Simulate

Visualize

Minimize & Visualize

Specifications *.mcrl2

```
act
  action1, action2, ...;
  action3, action4 : Type;

proc
  P1 = ...;
  P2(x: Bool) = ...;
      % Process expression

init
  SmUni;
```

```
sort List = struct
       empty | cons(A,List);

map sum2: Int # Int -> Int;

var x, y: Int;

eqn
  sum2(x,y) = (x+y) * (x+y);
  % Data patterns &  expressions
```

https://mcrl2.org/web/user_manual/language_reference/index.html

Process Expressions

$$P = \textbf{\textit{PE}} \ ;$$

`a`	*Action*
`a|b`	*Multi-action*
`P`	*Process*
`delta`	*Deadlock*
`a(`*DataExpr*`)`	*Parameterized Act.*
`P(`*DataExpr*`)`	*Parameterized Proc.*
`a.PE`	*Sequencing*
`PE1+PE2`	*Choice*
`PE1||PE2`	*Parallel*

`block({a,b},PE)`	*Block*
`allow({a,b},PE)`	*Allow*
`rename({a->b},PE)`	*Rename*
`comm({a|b->c},PE)`	*Communicate*
`sum m: Nat . PE`	*Gen. Choice*
exp `-> PE1 <> PE2`	*If-then-else*

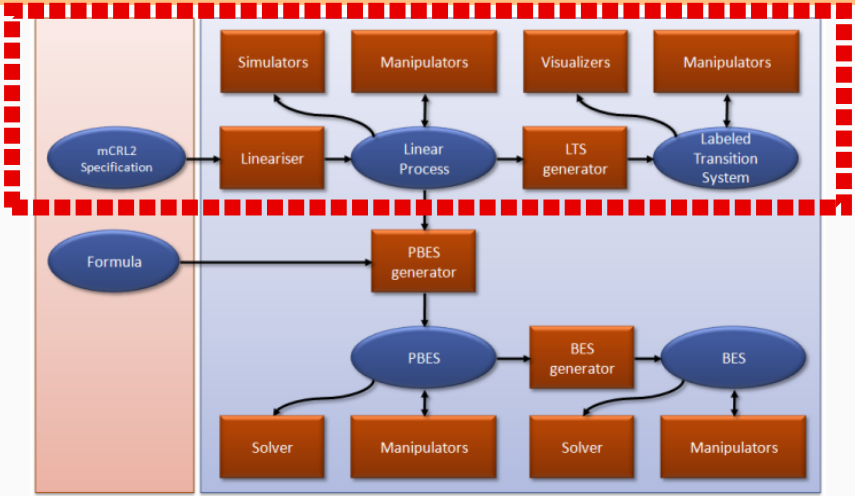
Data Expressions

$P(exp)$

`true`	*Boolean*
`42`	*Pos, Nat, Int, Real*
exp`!`	*Not*
exp `&&` *exp*	*And*
exp `||` *exp*	*Or*
exp `=>` *exp*	*Implies*
`forall n:Nat .` *exp*	*For all*
`exists n:Nat .` *exp*	*Exists*

exp `+` *exp*	*Sum*
`max(`*exp*`,`*exp*`)`	*And*
exp `mod` *exp*	*Remainder of div.*
`[`*exp*`,`*exp*`, ...]`	*List*
`{`*exp*`,`*exp*`, ...}`	*Set*
`{`*exp*`:2,`*exp*`:1, ...}`	*Bag*
`lambda n:Nat .` *exp*	*Function*

mCRL2 toolset overview



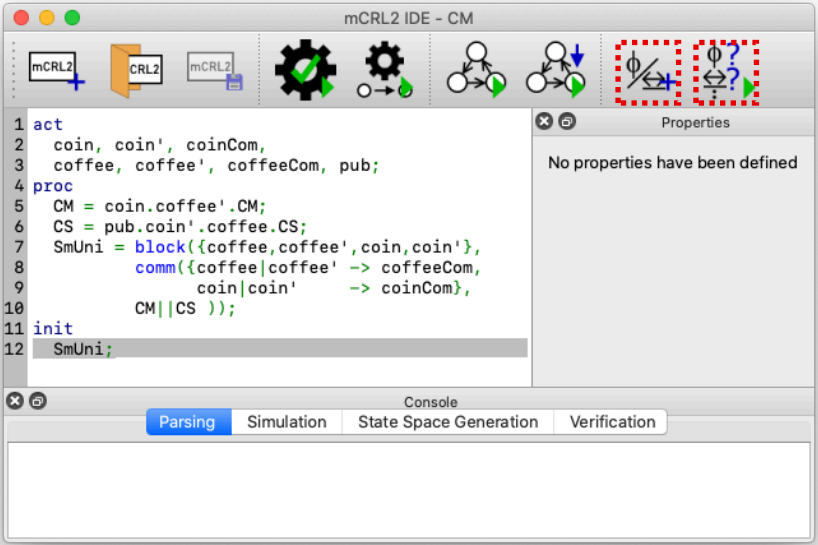
https://fm-dcc.github.io/sv2627/exercises/mcrl2-assignment.pdf

mCRL2 tutorial

https://fm-dcc.github.io/sv2627/exercises/adventurers/adventurers-tutorial-mcrl2.zip

Logic and Verification

mCRL2 IDE



Add properties

Verify properties

mCRL2 - modal logic

Syntax (simplified)

$$\phi = \texttt{true} \mid \texttt{false} \mid \texttt{forall x:T.}\phi \mid \texttt{exists x.:T}\phi$$
$$\mid \phi \; \mathit{OP} \; \phi \mid \texttt{!}\phi \mid \texttt{[}\mathit{expr}\texttt{]}\phi \mid \texttt{<}\mathit{expr}\texttt{>}\phi \mid \ldots$$

$$\mathit{expr} = \alpha \mid \texttt{nil} \mid \mathit{expr}\texttt{+}\mathit{expr} \mid \mathit{expr}\texttt{.}\mathit{expr} \mid \mathit{expr}\texttt{*} \mid \mathit{expr}\texttt{+}$$

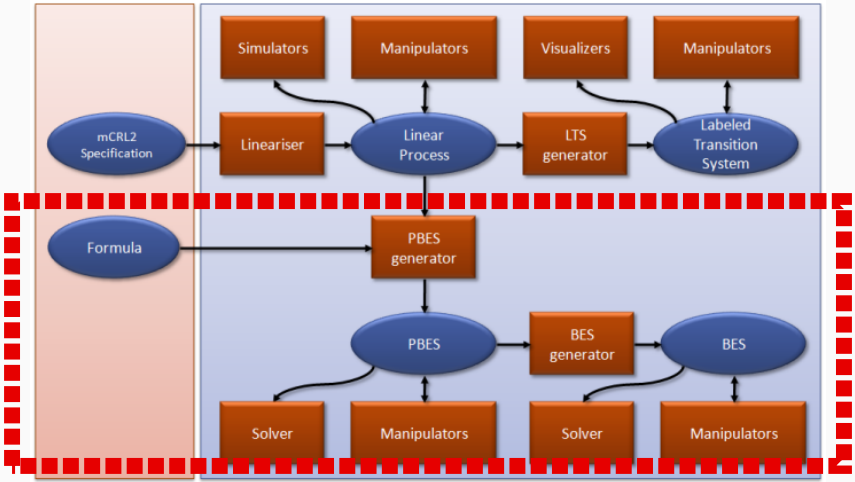
$$\alpha = \texttt{a(d)} \mid \texttt{a|b|c} \mid \texttt{true} \mid \texttt{false} \mid \alpha \; \mathit{OP} \; \alpha \mid \texttt{!}\alpha$$
$$\mid \texttt{forall x:T.}\alpha \mid \texttt{exists x:T.}\alpha \mid \ldots$$

where $T = \{\mathit{Bool}, \mathit{Nat}, \mathit{Int}, \ldots\}$ and $\mathit{OP} = \{\texttt{=>}, \texttt{\&\&}, \texttt{||}\}$

Example

"`[true*.a]<b>true`" means: *whenever an 'a' appears after any number of steps, it must be immediately followed by 'b'.*

mCRL2 toolset overview



Assignment 1 (now everything):

https://fm-dcc.github.io/sv2627/exercises/mcrl2-assignment.pdf